Ders 6 Genişletilmiş Notu: Nesneye Yönelik Çözümleme, Etkinlik ve Durum Şemaları, Tasarım Kalıpları

Ders 6 Genişletilmiş Notu: Nesneye Yönelik Çözümleme, Etkinlik ve Durum Şemaları, Tasarım Kalıpları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Çözümleme (Analiz) Nedir?
- 2. Uygulama Alanının Çözümlenmesi ve Gereksinim Analizi Adımları
- 3. Kullanım Öyküsü Belgesi: Market Kasası Örneği
- 4. Kullanım Şeması (Use Case Diagram) ve İlişkileri
- 5. Aday Sınıfların Bulunması ve Kavramsal Sınıf Şeması
- 6. SRS Belgesi: Analizden Tasarıma Geçiş
- 7. Ardışıl Şema (Sequence Diagram)
- 8. Etkinlik Şeması (Activity Diagram)
- 9. Durum Şeması (State Diagram)
- 10. Satış Uygulaması İçin Durum Şeması ve Tasarım Sınıf Şeması
- 11. Tasarım Kalıpları (Design Patterns)
- 12. Dokuz Temel Kalıp
- 13. Kalıpların Ardışıl Şemada Gösterilmesi: Kahve Sitesi
- 14. Etkileşim Şemaları ve Kahve Sitesi Sınıf Şeması
- Örnek Sorular

1. Çözümleme (Analiz) Nedir?

Hoca derse, önceki haftalarda nesneye yönelik analiz sürecinden, ondan önce de kullanım senaryosu (use case) ve alan modeli (domain model) tanımlarıyla analiz sürecinden bahsedildiğini hatırlatarak başladı. Nesneye yönelik analize geçmeden önce çözümlemenin ne olduğunu bir kez daha tanımladı.

Çözümleme (analysis), bir şeyi anlayabilmek için onu parçalara ayırmaktır. Sistemi anlamaya yönelik çalışmalar çözümleme ile tasarlanır. Bu aşamada uygulama alanının, yani problem alanının anlaşılması hedeflenir:

- Bu alanda hangi dil kullanılıyor?
- Hangi kavramlar var?
- Hangi çalışanlar var?
- Hangi işlemler yapılıyor?

Bu sorular cevaplanınca kullanıcıların gereksinimleri anlaşılmış olur. Ardından sınıflar, nesneler ve bunlar arasındaki etkileşim belirlenir ve bir çözümleme modeli oluşturulur. Hocanın özetiyle: Problemi çözebilmek için önce problemi anlamak gerekir; çözümleme aşaması bunu sağlar.

2. Uygulama Alanının Çözümlenmesi ve Gereksinim Analizi Adımları

Uygulama alanının çözümlenmesinde (domain analysis) hedef, uygulama alanını anlamak ve elde edilen bilgileri analiz modeline taşımaktır. Hoca alan modeli için daha önce Larman'ın orijinal bir slaytını kullandığını hatırlattı.

Hoca, daha önceki derslerde konuşulan gereksinim analizi adımlarını nesneye yönelik analize aktardı. Gereksinim analizinde genel olarak 7 adım vardı. Bu derste bunların ilk dördü üzerinde duruldu:

- **İşi anlama (giriş, inception):** İlk adımda bu işteki hedefimizi anlarız.
- **Bilgi toplama (elicitation):** Uygulama alanı hakkında bilgi toplanır. Mevcut uygulamalar nelerdir, teknik literatür nedir, müşteri anketleri ne söylüyor, mevcut ve gelecekteki gereksinimler neler olabilir? Bu bilgiyi edinmenin temel nedeni, müşterinin dilinden anlayabilmek ve müşterinin dediklerini anlamak için ön bilgi oluşturmaktır.
- **Modelleme (elaboration):** Kullanım şemaları, sınıf şemaları, kavramsal sınıf şemaları gibi sistemi anlamaya yönelik modeller oluşturulur.
- **Pazarlık (negotiation):** Modeller gözden geçirilir ve taraflar uzlaşır.

  Ek bilgi (derste anlatılmadı): Hocanın "7 adım" dediği yapı büyük olasılıkla Pressman'ın gereksinim mühendisliği adımlarıdır: inception, elicitation, elaboration, negotiation, specification (belirtim), validation (doğrulama) ve requirements management (gereksinim yönetimi). Bu derste ilk dördü anılmış, belirtim adımının çıktısı olan SRS belgesine ise ayrıca değinilmiştir.

2.1 Pazarlık Aşaması

Hoca pazarlık aşamasında ne yapıldığını sınıfa sordu. Bir öğrencinin cevabı ve hocanın eklemeleriyle şu tablo ortaya çıktı:

- Müşterinin farklı birimleri vardır; örneğin tasarım ekibi ve muhasebe ekibi. Bu birimlerin talepleri birbiriyle uyuşmayabilir ya da aynı olayı farklı anlatabilirler.
- Bazı istekler maliyeti çok yüksek olduğu ya da uygulanabilirliği düşük olduğu için yapılamaz.
- Taraflar iletişim kurar. Bazı kısımlardan vazgeçilir, bazı kısımların farklı biçimde yapılmasında uzlaşılır.
- Pazarlık yalnız müşterinin kendi birimleri arasında değil, müşteri tarafı ile yazılım geliştirici tarafı arasında da olur. Müşterinin isteyip de yapılamayacak şeyler çıkarılır.

Hoca bu cevabı onayladı: Pazarlık aşaması, yapılamayacak isteklerin çıkarılması ve müşterinin farklı birimlerinin olayı farklı anlatmasından doğan çatışmaların önlenmesidir.

3. Kullanım Öyküsü Belgesi: Market Kasası Örneği

Alan modeli oluşturulduktan sonra model aşamasına geçilir. Gereksinimler belirlenirken önce kullanıcı hikâyesi alınır. Müşterinin beklentileri doğal dilde bir belge olarak not edilir, sonra bunun modeli çıkarılmaya çalışılır. Bu belgelerden iki şey üretilir:

- Kullanım öyküsünün belge tarafı (use case belgesi).
- Kullanım şeması (use case diagram).

3.1 Belgenin Bölümleri

Geçen hafta bakılan kullanım öyküsü formatında şu bölümler vardır:

- **Birincil aktör:** Market örneğinde kasiyer.
- **İlgililer ve ilgi alanları:** Kasiyerden başka satıcı ve müşteri. Birincil aktör kasiyerse diğerlerine ikincil aktör ya da ilgililer denebilir. Her birinin hedefi belirtilir.
- **Ön koşul:** İşin başlaması için gereken koşul.
- **Son koşul:** İşin "bitti" denebilmesi için sağlanması gereken koşul.
- **Ana akış:** Öykünün adımları.
- **Alternatif akışlar:** Normal koşullar sürmezse ne olacağı.

> **Hocanın vurgusu:** Kullanım öyküsünde yer alacak her şey ilgi alanlarına giren şeyler olmalıdır. Bizden istenenlerin dışındaki şeyler hakkındaki bilgiyi use case belgesine koymamalıyız.

Hoca aktörü yeniden tanımladı: Aktör, sistemle etkileşimde bulunan varlıktır. Bu bir insan olabileceği gibi bir yazılım ya da donanım da olabilir.

3.2 Ana Akış ve Alternatif Akışlar

Ana öykü, algoritma yapısından bildiğimiz gibi işin adım adım nasıl gerçekleştirileceğini gösterir: neler yapılıyor, neler yapılması gerekiyor ve hangi sırada yapılması gerekiyor. Satış işleminde işin sırası önemlidir. Örnekte ana akış 11 adımdan oluşuyordu.

Normal koşullar sürmezse temel öykünün içinde alternatif öyküler yer alır. Hocanın verdiği iki örnek:

- **Geçersiz barkod:** Kasiyer ürünü okuttuğunda barkod geçersizse sistem uyarı mesajı verir, kayıt girişini reddeder ve yeniden okunabilir bir ürün getirilmesi istenir.
- **Müşteri bir ürünü bırakmak ister:** Normal öyküde 7. adımda kasiyer müşteriye toplamı bildirir ve ödeme yapmasını ister. Bu adımda müşteri aldıklarının fazla geldiğini düşünüp bir ürünü çıkarmak isteyebilir. Kasiyer o ürünün barkodunu okutur, ürün toplamdan düşülür ve güncel toplam bildirilir.

Kullanım öyküleri bu şekilde yazıldıktan sonra, ya da yazılmadan önce, bir taslak şema çizilir.

4. Kullanım Şeması (Use Case Diagram) ve İlişkileri

Hoca kullanım şemasının sembollerini tekrarladı:

Sembol	Anlamı
Elips	İşlem, yani kullanım senaryosu
Çöp adam	İnsan aktör (kullanıcı)
Dikdörtgen	Sistemle ilişkili insan olmayan aktör: donanım (örneğin bir algılayıcı, sensör) ya da başka bir yazılım
Düz çizgi	Aktör ile senaryo arasındaki etkileşim
Kesikli ok	Senaryolar arasındaki ilişki (`<<include>>`, `<<extend>>`)

4.1 Include ve Extend

Hoca senaryolar arasındaki ilişkileri sınıfa sordu. Bir öğrencinin cevabı şöyleydi:

- **Extend (genişletme):** A işlevini yürütürken, istenirse B işlevine sapılması gerektiğini belirtir. Kesikli okla gösterilir.
- **Include (içerme):** A işlevini yapabilmek için B işlevinin de gerçekleştirilmesi gerektiğini belirtir.

Hoca bunu iki örnekle pekiştirdi. Kitaptaki kahve sitesi örneğinde sipariş vermek, daha önce siteye bilgi girişi yapılmış olmasını gerektirir. Yani sipariş vermek bilgi girişini içerir; ikisi iç içedir.

İkinci örnek market kasasıydı. Kasiyer aktörü satış senaryosunda görev alır. Satışın yapılması, ödemenin yapılmasını da kapsar; bu bir include ilişkisidir. Ödeme yapıldığında bazen provizyon işlemi de gerekir, yani kredi kartının limitinin aşılıp aşılmadığı gibi bilgilerin sınanması. Ama ödeme nakitse provizyona hiç gerek kalmaz. Provizyon her zaman yapılan bir işlem değildir; gerektiğinde yapılan bir genişletmedir. Bu bir extend ilişkisidir.

```
Kasiyer ----------- (Satış Yap)
(Satış Yap)    - - <<include>> - -> (Ödeme Al)      her satışta
(Provizyon Al) - - <<extend>>  - -> (Ödeme Al)      yalnız kartla ödemede
```

> **Ek bilgi (derste anlatılmadı):** UML gösteriminde include oku temel senaryodan içerilen senaryoya, extend oku ise genişleten senaryodan temel senaryoya doğru çizilir. Yukarıdaki metin şemada oklar bu kurala göre yazılmıştır.

5. Aday Sınıfların Bulunması ve Kavramsal Sınıf Şeması

Analiz sırasında sınıflar da belirlenir. Sınıflar kullanıcı gereksinimleri belgesinden ve kullanım senaryolarından elde edilir. Önce kullanım senaryosu, sonra kullanıcı gereksinim belgesi oluşturulabilir.

Bu belgelerdeki isimler taranarak aday sınıflar bulunur. Bir aday, şu kurallardan en az birini sağlamalıdır:

- Saklanan bir bilgi olmalıdır.
- Gereksinim duyulan bir hizmet olmalıdır.
- Problemin çözümü için gerekli bir varlık olmalıdır.

Her sınıfın kullanım senaryosu içinde bir görevi olmalı ve bu üç türden biri olmalıdır.

5.1 Sorumluluklar ve CRC Kartları

Sınıflar belirlenirken işlemler ve eylemler tanımlanır, sonra her birinin sorumluluğu belirlenir. Sorumluluklar sınıf–sorumluluk–işbirliği kartlarıyla (Class Responsibility Collaboration, CRC) belirlenir ve sınıflara dağıtılır.

Hoca etkileşimi şöyle tanımladı: Bir nesnenin üzerine düşen sorumluluğu yerine getirmek için başka bir nesneye mesaj göndermesi bir etkileşimdir.

5.2 Kavramsal Sınıf Şeması: Market Örneği

> **Hocanın vurgusu:** Kavramsal sınıf şemalarında sınıflara uygulanacak mesajlar üzerinde durulmaz. Yalnız sınıfların adları, özellikleri (nitelikleri) ve birbirleriyle ilişkileri gösterilir.

Örnek modelde satış kalemi ile satış arasında bir ilişki vardır. Bu ilişkinin bir toplama (aggregation) mı yoksa bileşim (composition) mi olduğuna bakılır:

- **Aggregation:** İçi boş eşkenar dörtgenle gösterilir.
- **Composition:** İçi dolu eşkenar dörtgenle gösterilir.

Hoca sınıfları tek tek açıkladı:

- **Satış:** Satış işinin bilgilerini içerir. Özellikle satışın hangi tarihte ve saatte yapıldığı, faturanın hangi tarihte çıkacağı.
- **Satış kalemi:** Kasadan bir kerede geçen her ürün tipi bir kalemdir. Kalem, o tipten kaç tane alındığını tutar.
- **Ödeme:** Ödeme miktarını tutar. Ödeme kredi kartıyla da olabilir, nakit de.
- **Kasa:** Bir donanım öğesidir.
- **Şube:** Birden fazla şube olabilir ve bir şubede birden fazla kasa olabilir.
- **Ürün:** Hangi satış kaleminin hangi ürünü içerdiği bu ilişkiyle tanımlanır.

Satış kalemi ile ürün ilişkisi için hocanın örneği: Bir defter alınmışsa satış kalemindeki miktar 1'dir. Aynı defterden beş tane alınmışsa yine tek bir satış kalemi olur, miktarı 5'tir. Bir kalemde en az bir ürün bulunur, daha fazlası da olabilir.

Bu modelde sınıf kutularının üçüncü bölümü boştur. Sınıflar arasındaki işlemleri tetikleyen mesajlara dair bilgi yoktur, çünkü bu bir kavramsal modeldir.

6. SRS Belgesi: Analizden Tasarıma Geçiş

Çözümleme sürecinde modelleme yapıldıktan sonra modeller pazarlık aşamasında bir kez daha gözden geçirilir. Taraflar uzlaşınca yazılım gereksinimleri belirtimi (Software Requirements Specification, SRS) belgesi elde edilir.

SRS belgesinde hangi uygulamanın gerçekleştirildiği gibi formal bilgiler de bulunur. Şemalar ve belgeler açısından bu belgede şunlar olmalıdır:

- UML kullanım şemaları.
- Kullanım senaryoları.
- UML sınıf şemaları.
- Veritabanı için varlık-ilişki (ER) şemaları.

Bunların hepsi analiz raporunun, yani analiz belirtiminin içindedir. SRS belgesi yazılım gereksinim analizi aşamasının belgesidir. Bundan sonra bu belge esas alınarak tasarıma geçilir.

7. Ardışıl Şema (Sequence Diagram)

Tasarımda ilk kullanılan şemalardan biri ardışıl şemadır. Hoca bu şemaya sıralama şeması ya da ardışıl şema dendiğini, hepsinin aynı işlevi gördüğünü söyledi.

- Nesneler şemanın üstünde sıralanır. Bu sıra rastgele değildir; nesneler işleme giriş sırasıyla dizilir.
- Altta mesajlar ve metot çağırma adımları gösterilir.
- Bir işlemin ya da bir nesnenin işleme girişinin hangi koşulda başlayacağı da gösterilebilir.
- Bir nesnenin kendi kendini çağırması da gösterilebilir.

Bu şemalar geçen hafta kitaptaki kahve satış sitesi örnekleriyle işlendiği için hoca üzerinde fazla durmadı. Nesneye dayalı kavramlar dersinde de bu konunun işlendiğini, o dersi almayanların Larman'ın kitabını ya da o sitedeki bilgileri daha ayrıntılı incelemesi gerektiğini söyledi.

8. Etkinlik Şeması (Activity Diagram)

Etkinlik şeması, sistem içinde gerçekleşen olayların farklı mekânlarda ve farklı ortamlarda nasıl yürüdüğünü görmek için çok uygundur. Sistemin analizi yapılırken kullanılmaya başlanır. Tasarım sırasında da etkinlik şemaları ile ardışıl şemalar birbirini tamamlar ve birlikte sistem içindeki yapılanmanın nasıl olacağını gösterir.

> **Hocanın vurgusu:** Etkinlik şemasını analizde çok kabaca kullanmaya başlarız. Tasarımdaki ayrıntıları ise etkinlik şeması ve ardışıl şemalarla tamamlarız.

8.1 Semboller

Sembol	Anlamı
Ok	Denetim akışı (etkinlikler arasındaki geçiş)
Dolu yuvarlak	Etkinliğin başlangıcı
Olay kabul etme / sinyal alma işareti	Başlangıçta bir olayın kabul edilmesi, sinyal alınması
Daire içinde içi dolu yuvarlak	Etkinliğin sonu
Kenarları yuvarlatılmış dikdörtgen	Eylem (etkinlik)
Çatal (fork)	Akışın aynı anda yürüyen iki kola ayrılması
Birleşme (join)	Kolların birbirini bekleyip tek akışta birleşmesi
Sütunlar (kulvar, swimlane)	Her sütunda bir aktörün yaptığı eylemler

> **Ders sonrası not:** Hoca eylem sembolü için önce "dikdörtgen" dedi, hemen ardından "veya kenarları yuvarlak" diye düzeltti. UML'de eylem kutusu kenarları yuvarlatılmış dikdörtgendir.

8.2 Uçak Yolculuğu Örneği

Bu bölümde dersin bağlantısı bir kez kesildi; hoca yeniden ekran paylaşıp örnek şema üzerinden devam etti. Örnek bir uçak yolculuğundaki işlemlerdi. Metin olarak akış şöyledir:

Sıra	Öğe	Açıklama
1	Başlangıç	Başlangıç sinyalinin alınması
2	Eylem	Check-in işlemi
3	Karar	Check-in başarılı mı?
3a	[başarısız]	Bu kol etkinlik sonu işaretiyle biter
3b	[başarılı]	Akış devam eder
4	Çatal (fork)	Akış iki ortama ayrılır
4a	Kol 1	Yolcunun uçağa binişi
4b	Kol 2	Bagajların uçağa yüklenmesi
5	Birleşme (join)	İki işlem de bitince aynı noktada birleşilir
6	Eylem	Uçağın kapılarının kapanması, uçağın piste doğru ilerlemesi

Etkinlik şemasının asıl faydası buradadır: Uygulamadaki akışları farklı ortamlarda takip etmek istediğimizde bir tarafta yolcunun uçağa binmesini, diğer tarafta bagajların yüklenmesini aynı anda gösterebiliriz. Çatal, iki farklı ortamda aynı zamanda hangi etkinliklerin olduğunu gösterir. Birleşme ise iki kolun da bitmesini bekler; ikisi de bitince akış aynı noktadan devam eder.

Bu noktada bağlantı ikinci kez kesildi. Hoca döndüğünde aynı yapıyı bir kez daha özetledi: Fork ve join yapısı, işlemlerin iki ayrı mekânda, iki ayrı ortamda nasıl yürüdüğünü gösterir. Eylemler aynı düzende başlar, birbirini bekler ve bitiş noktasından sonra aynı noktada devam eder.

8.3 Kulvarlı Örnek: Birden Fazla Aktör

İkinci örnek, birden fazla aktörü ve aktörler arasındaki bilgi akışını gösteriyordu. Aktörler yer hizmetleri çalışanları ve müşteri hizmetleri çalışanlarıdır:

Kulvar	Eylemler
Yer hizmetleri çalışanları	Uçağa binenlerin listesinin oluşturulması
Müşteri hizmetleri çalışanları	Yolcu bilgilerinin “bunlar bizim müşterimizdir” diye sürekli olarak veritabanına kaydedilmesi

Etkinlikler yine kenarları yuvarlatılmış dikdörtgenlerle gösterilir. Şema sütunlara bölünür. Her sütunda bir aktörün etkili olduğu işlemler, eylemler ve bu eylemlerin birbirleriyle ilişkileri tanımlanır. Böylece hangi aktörün hangi işlemleri yaptığı ve işin nasıl yürüdüğü görülür.

9. Durum Şeması (State Diagram)

Tasarım şemalarından biri de durum şemasıdır. Hoca etkinlik şemasıyla farkını şöyle koydu: Etkinlik şemasını farklı ortamlarda, farklı aktörlerin bulunduğu yerlerde etkinliklerin nasıl yürüdüğünü izlemek için kullanıyorduk. Durum şeması ise sistemin hangi durumda olduğunu, hangi noktadan hangi noktaya gittiğini gösterir.

Durum şeması şu amaçlarla kullanılır:

- Bir aktörün ya da bir işlemin şu anda hangi durumda olduğunu göstermek.
- Bir aktörün, sürecin başlangıcından sonuna kadar hangi durumlarda bulunduğunu takip etmek.
- Durum geçişleri sırasında hangi işlemlerin gerçekleştirildiğini belirtmek.

Çizim kuralları:

- Bir başlangıç aşaması vardır.

- Kenarları yuvarlatılmış kareler, aktörün o anda hangi durumda olduğunu gösterir.
- Durumlar arasındaki bağlantı okları, hangi eylemlerin gerçekleştirildiğini gösterir.

9.1 Uçağın Yaşam Döngüsü

Havacılık örneğinden gidildiği için hoca bir uçağın yaşam döngüsünde hangi durumlardan geçtiğini gösterdi:

Durum	Açıklama
Sipariş	Uçağın alım emri verildi; satın alınacak firmadan sipariş gerçekleştirildi
Teslim	Teslimat yapıldı
Bakım	Uçağın hareket etmesi için arıza olup olmadığına bakılır
Kullanıma hazır	Bakım tamamlandı
Kullanımda	Kullanım süreci devam ediyor
Kullanım dışı	Uçağın kullanım süresi bitti, artık kullanılmayacak

Geçişler: Sipariş → Teslim → Bakım → Kullanıma hazır → Kullanımda. Kullanım sırasında arıza bulunursa uçak yeniden bakıma girer; bakım tamamlanınca tekrar kullanıma hazır olur. Kullanım süresi bitince kullanım dışı durumuna geçer.

Hoca bunu genelleştirdi: Herhangi bir malzeme ya da donanım için süreçte hangi durumlarda bulunduğunu gösteren bir şema çizilebildiği gibi, bir uygulama içinde de durum geçişleri bu şekilde gösterilebilir.

10. Satış Uygulaması İçin Durum Şeması ve Tasarım Sınıf Şeması

Hoca, daha önce kullanım senaryosu ve alan modeli oluşturulan market satış uygulaması için bir tasarım modeli kurmaya geçti: bir durum şeması ve bir sınıf şeması.

10.1 Kasa Durum Şeması

Şema kasanın durumlarını gösterir:

Durum	Ne oluyor?
Boşta bekleme	Kasada herhangi bir müşteri yok
Satışın kaydedilmesi	Satış başlatıldı; barkodlar giriliyor, satıştaki işlemler yapılıyor
Satıştan tek ürün silme	Alt durum: barkod giriliyor ve ürün iptal düğmesine basılıyor
Ödeme bekleniyor	Alınacak bütün ürünler kaydedildi, müşteriden ödeme bekleniyor
Fiş / fatura / makbuz verme	Ödeme alındı

Geçişler:

Kaynak durum	Olay / koşul	Hedef durum
Başlangıç	—	Boşta bekleme
Boşta bekleme	Satış başlatıldı	Satışın kaydedilmesi
Satışın kaydedilmesi	Ürün iptal (barkod + iptal düğmesi)	Satıştan tek ürün silme
Satıştan tek ürün silme	Silme tamamlandı	Satışın kaydedilmesi
Satışın kaydedilmesi	Bütün ürünler kaydedildi	Ödeme bekleniyor

Kaynak durum	Olay / koşul	Hedef durum
Ödeme bekleniyor	Ödeme alındı	Fiş / fatura verme
Fiş / fatura verme	—	Boşta bekleme (yeni müşteri için döngü)
Boşta bekleme	Kasa kapatma düğmesine basıldı	Bitiş

Tek ürün silme durumu, kullanım öyküsü belgesindeki “müşteri bir ürünü bırakmak ister” alternatif akışının karşılığıdır. Her yeni müşteri için bu durumların hepsi yeniden yaşanır; bu döngü kasa kapatma düğmesine basılana kadar sürer.

Hoca bu tasarım modelini, kişi ya da donanım aktörlerinin durumlarını an be an takip etmeye yarayan faydalı bir araç olarak niteledi.

10.2 Tasarım Sınıf Şeması

Kavramsal sınıf şemasında işlemler yoktu. Tasarım sınıf şeması, UML sınıf şemasının tam hâlidir. Kavramsal şemaya göre bazı düzeltmeler ve farklılıklar içerebilir ama genel olarak aynıdır. Sınıflar ve sınıflar arası ilişkiler korunur; asıl fark, sınıf kutusunun üçüncü bölümünün dolmasıdır. Burada hangi eylemin tetikleneceğiyle ilgili mesajlar, yani sınıfın gerçekleştireceği işlemler tanımlanır.

Hocanın verdiği örnek işlemler:

- Ürünün var olup olmadığını döndüren, mantıksal (boolean) değer veren bir işlem.
- Ödeme kalemlerini veren bir işlem.
- Satışın yapıldığı tarihin alınması.
- Toplamın hesaplanması.
- Her kasada barkodların okunması; kasada bu özelliğin olması gerekir.

Hoca, sınavdan önceki hafta araştırma görevlilerinin bütün şemaların oluşturulduğu bir uygulama dersi yapacağını söyledi.

11. Tasarım Kalıpları (Design Patterns)

Aradan sonra hoca yeni konuya geçti: Nesneye yönelik yaklaşımda tasarım denince akla tasarım kalıpları gelir. Hoca tasarım kalıbının kelime anlamını “aklın yolu birdir” diye özetledi.

Slayttaki kırmızıyla belirtilmiş tanımlar şunlardı:

- Nesneye yönelik tasarımda genel prensipler ya da akla yakın çözümler bir araya getirilerek bir repertuvar oluşturulmuştur.
- Sıklıkla karşılaşılan, tanımlanabilir bir probleme sistematik bir çözüm getiren prensiplere tasarım kalıbı denir.
- Belirli bir problemi yeniden düşünmek yerine kalıbın çözümü uygulanır.
- Her tasarım kalıbına, o problemi akla getiren bir isim verilir.
- Kalıplar, tanımı yapılmış bir problemin en iyi çözümünü sağlar ve temel yolları tarif eder.

Hoca başlangıçta 23 temel kalıp tanımlandığını, sonradan bunlara eklemeler yapıldığını söyledi. Bu kalıpları ilk ortaya atan dört yazılım uzmanı Gamma, Helm, Johnson ve Vlissides’tir; literatürde bunlara “Dörtlü Çete” (Gang of Four, GoF) kalıpları denir. Hoca bu kalıpların genel olarak sorumluluk atama gibi temel işleri yaptığını ve her kalıbın bir sorunun cevabını verecek şekilde yaratıldığını söyledi.

> **Ders sonrası not:** Hoca derste anlattığı dokuz kalıbı (yaratıcı, bilginin uzmanı, az bağımlılık, denetçi, yüksek uyum, çok şekillilik, yapay sınıf, dolaylılık, değişimlerden korunma) GoF’un 23 kalıbının en çok kullanılan dokuzu olarak sundu; bir yerde de “Larman’ın ilk 23 kalıbı” dedi. Doğrusu: Bu dokuz ilke Craig Larman’ın *Applying UML and Patterns* kitabındaki **GRASP** (General Responsibility Assignment Software Patterns) ilkeleridir. GoF’un 23 kalıbı ise Gamma, Helm, Johnson ve Vlissides’in 1994 tarihli *Design Patterns* kitabındadır (Singleton, Factory Method, Adapter, Observer, Strategy gibi). Hocanın “sorumluluk atama” vurgusu GRASP’a uyar.

11.1 Yeniden Kullanılabilirlik

Hoca kalite özelliklerine henüz girilmediğini, ama yeniden kullanılabilirlik (reusability) kavramının başka derslerde karşılaşılmış olabileceğini söyledi. Yeniden kullanılabilirlik, yazılımda kaliteyi artıran bir kalite özelliğidir. Bir yazılımın bütünü, bir parçası ya da bir modülü başka uygulamalarda da kullanılabilecek şekilde genelleştirilerek tasarlanıp yazılmışsa yeniden kullanılabilirlikten söz edilir. Tasarım kalıplarıyla en çok sağlanmaya çalışılan özellik budur.

Tasarım kalıplarının kullanımı kaliteyi artırır. Kalite arttığı için yazılım bakımı da iyileşir. Sorumluluklar doğru atanmışsa sistem daha anlaşılır olur ve bakımı kolaylaşır.

12. Dokuz Temel Kalıp

Hoca en çok kullanılan dokuz kalıbı tanımladı ve bunları anlayıp Larman'ın kitabındaki örnekleri takip etmeyi önerdi.

> **Hocanın vurgusu:** Size bir tasarım veririz. Bu tasarımın içine tasarım kalıplarını da yerleştirmeye çalışarak cevabınızı oluşturun.

12.1 Yaratıcı (Creator)

Bir sınıftan nesne yaratmak çok sık karşılaşılan bir iştir. Soru şudur: Bu sorumluluk kime verilecek, sınıftan nesneyi kim yaratacak? Yaratma işi yaratıcı kalıbına uygun yapıldığında tasarımda kalite sağlanır: daha az bağımlı, daha anlaşılır ve yeniden kullanılabilir nesneler elde edilir.

Kahve sitesi örneğinde sipariş kalemi nesnesini sipariş yaratır. Bu kalıp, ardışıl şemalarda nesne tanımları yapılırken işin destekçisi olarak şemaya eklenir.

12.2 Bilginin Uzmanı (Information Expert)

Altyazıda "ekspert kalıbı" diye de geçen bu kalıp, nesnelere sorumluluk atama prensibidir. Sınıftan nesneyi yarattık; peki nesnelere sorumlulukları nasıl atayacağız? Bir sorumluluk, o işi yapmak için gereken bilgiye sahip olan sınıfa, yani bilginin uzmanına atanır. Sorumluluk atanırken öncelikle bu kalıba bakılır.

Kahve sitesi örneği:

- Bir siparişin toplam bedelini bilme sorumluluğu için en iyi sınıf sipariş sınıfıdır.
- Her sipariş kalemi kendi bedelini bilme sorumluluğunu taşır.
- Sipariş kalemi kendi bedelini hesaplamak için kahve nesneleriyle işbirliği yapmak zorundadır.

Hangi sınıfın hangi sınıfla işbirliği yapacağı, CRC kartlarındaki "collaboration" kısmında burada tanımlanmış olur.

12.3 Az Bağımlılık (Low Coupling)

Az bağımlılık, diğer sınıflarda olabilecek değişimlerden etkilenmemeyi ve yeniden kullanılabilirliği sağlamakla görevli kalıptır. Bağımlılık, sınıfların birbirine ne kadar bağlı olduğudur. Birbirine çok fazla değişken göndermek ve almak, çok bağımlılığı gösterir.

Fazla bağımlılığın zararları:

- Bakımı zorlaştırır ve test işini zorlaştırır.
- Bir sınıftaki değişim, bağımlı olduğu diğer sınıfları da etkiler.
- Yeniden kullanım zorlaşır.

Sınıfları tamamen izole yapmak mümkün değildir, ama bağımlılık fazla olmamalıdır.

Kahve sitesi örneğinde hem sipariş hem ödeme müşteriyle ilişkilidir. Yaratıcı kalıbına göre ödemeyi müşterinin yaratması gerekir; ancak o zaman müşterinin ödeme hakkında bilgiye sahip olması gerekir. Bu sorumluluk sipariş sınıfına atanırsa bağımlılık azaltılmış olur.

Buradan çıkan ders: Sınıfların yaratılması, sorumluluğun atanması ve bağımlılığın azaltılması birlikte düşünülerek tasarım yapılmalıdır. Kalıplar birlikte kullanılarak en iyi çözüm sağlanır.

> **Ek bilgi (derste anlatılmadı):** Larman'ın kitabındaki asıl örnek market kasası (POS) sistemidir: Yaratıcı kalıbı ödemeyi kasanın (Register) yaratmasını önerir, ama ödemeyi satışın (Sale) yaratması kasa ile ödeme arasındaki bağımlılığı ortadan kaldırdığı için tercih edilir. Hocanın kahve sitesi örneği büyük olasılıkla aynı akıl yürütmenin bu örneğe uyarlanmış hâlidir.

12.4 Denetçi (Controller)

Kullanıcı arayüzünün arkasında durarak sistem olaylarını alma ve kontrol etme sorumluluğunu üstlenecek bir sınıfa ihtiyaç vardır. Örneğin ekrandan bilgi alınırken bu bilginin kontrol edilmesi gerekir.

- Tüm sistemi ya da bir alt sistemi temsil eden bir sınıf denetçi olarak kullanılabilir.
- Her kullanım senaryosunu temsil eden bir sınıf denetçi olarak atanabilir.
- Denetçi nesneleri sistem olaylarını algıladıktan sonra kontrol işlerini yapar ve olayı esas işlemi gerçekleştirecek nesnelere yönlendirir.

Hoca, denetçinin uygulama kolaylaştırıcı özelliklerle karıştırılmaması gerektiğini söyledi. Bunlar yalnız sistem olaylarını ilgili denetçiye aktaran yapılardır.

12.5 Yüksek Uyum (High Cohesion)

Hoca yüksek uyum ile az bağımlılığı birlikte ele aldı: İyi bir tasarım için bunlar sağlanmalıdır. Bu kural yapısal (structural) tasarımda da geçerlidir; orada modüllerin birbirinden yeterince bağımsız, modül içinde ise kuvvetli olması istenir. Nesneye yönelik tasarımda da sınıf içindeki uyumun fazla, nesneler arasındaki bağımlılığın az olması iyi tasarımın ölçüsüdür.

> **Hocanın vurgusu:** İyi tasarım: yüksek uyum (high cohesion) ve az bağımlılık (low coupling).

Nesneye yönelik bir programda nesneler kendilerine atanan sorumlulukları yerine getirmekle yükümlüdür. Bir sınıfa atanan sorumlulukların birbiriyle ilgili olması ve belirli bir konuya yoğunlaşması beklenir. Bir sınıf çok fazla iş yapıyorsa ve sorumlulukları birbirinden farklıysa o sınıfta uyum kötüdür. Sınıf, anlaşılırlığı azaltılmadan ve yeniden kullanılabilirliği zorlaştırılmadan oluşturulmalıdır. Birlikte çalıştığı çok fazla sınıf varsa sınıf değişimlerden çok daha fazla etkilenir. Sorumluluklar sınıf içinde iyi bir uyum yakalanacak şekilde atanmalıdır.

12.6 Çok Şekillilik (Polymorphism)

Hoca çok şekilliliğin ne olduğunu sınıfa sordu ama cevap gelmedi. Kalıbın çözümünü şöyle verdi: Davranışın tipe göre değişebildiği problemlerin önüne geçebilmek için, sorumluluk davranışın değiştiği tiplere çok şekilli metotlar kullanılarak atanır. Nesneye yönelik bir yazılımın bileşenleri için bu yapı gereklidir. Sisteme belirli bir parçanın nasıl monte edileceği çok şekillilik göz önünde bulundurularak belirlenir.

Hoca, çok şekillilik tanımında eksiği olanların nesneye yönelik kavramlarla ilgili belgelere bakmasını istedi.

12.7 Yapay Sınıf (Pure Fabrication)

Bilginin uzmanı kalıbının getirdiği çözümler, iyi uyum ve az bağımlılık ilkeleriyle çelişebilir. Bu durumda, bağımlılıkları azaltıyorsa ve tekrar kullanılabilirliği artırıyorsa, gerçek dünyada var olmayan yapay sınıflar oluşturulabilir.

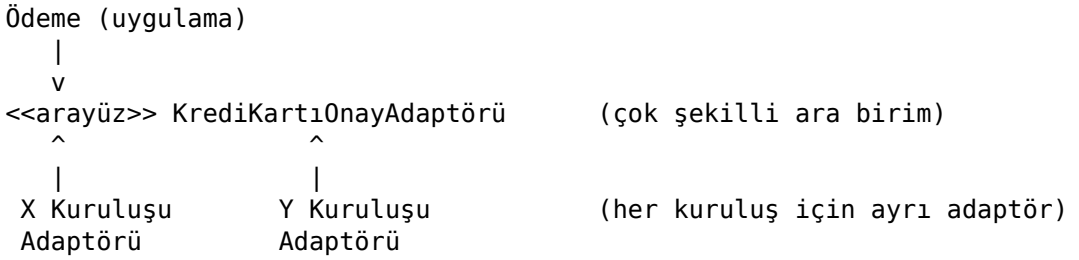
- Birbirine benzeyen işlevler bir sınıfta toplanır ve gerçek dünyada karşılığı olmayan bir sınıf yaratılır.
- Bu sınıf tasarlanırken gerçek dünya izlenir ve ona benzer bir yapı kurulur.
- Sorumluluk ataması, gerçek hayattaki normal durumlardan farklı olabilir; amaç uyum ve bağımlılık problemlerine çözüm getirmektir.

> **Hocanın vurgusu:** Yapay sınıf kalıbının fazla kullanılmaması önerilir. Çok fazla yapay sınıf kullanılırsa tasarım gerçek dünyadaki uygulamalardan uzaklaşır ve anlaşılırlık azalır.

12.8 Dolaylılık (Indirection)

Dolaylılık, ara birim kalıbıdır. Kolayca değişebilecek yapılar arasındaki bağımlılığı azaltır ve bu sayede yeniden kullanılabilirliği artırır. Değişebilecek iki birim, bileşen ya da servis arasındaki sorumluluklar bir ara birim nesnesine atanır; böylece iki yapı birbirine doğrudan değil, dolaylı olarak bağlanır. Eklenen ara birim çok şekillilik de getirir ve dış değişimlerden etkilenmemeyi sağlar.

Kahve sitesinden örnek: Ödeme tarafında kredi kartı onayı dışarıdan alınan bir hizmettir, yani başka bir yazılım tarafından sağlanır. Değişik kredi kartı kuruluşlarının değişik onay mekanizmaları vardır. Uygulama açısından yalnız "kredi kartı onayı" sorumluluğu vardır. Bunun üzerine çok şekilli bir ara birim nesnesi konur ve bu ara birimi gerçekleyen kredi kartı sistem adaptörleri kullanılır. Bir genelleştirme yapılarak kredi kartı adaptörü uygulaması dolaylılık kalıbı sayesinde gerçekleştirilmiş olur.



> **Ek bilgi (derste anlatılmadı):** Bu kredi kartı örneği GoF kalıplarından **Adapter** kalıbının tipik kullanımıdır: Farklı dış servislerin farklı arayüzleri, uygulamanın beklediği tek bir arayüze uyarlanır. GRASP'taki dolaylılık ve değişimlerden korunma ilkeleri, Adapter gibi GoF kalıplarının altında yatan genel fikirlerdir.

12.9 Değişimlerden Korunma (Protected Variations)

Hocanın andığı son tasarım ilkesi budur. Bir sistemdeki her alt sistemde, hatta her nesnede değişimler ve kararsızlıklar olabilir. Bunların diğer birimlere etkisini en aza indirmek için, değişime açık ve kararsız noktaların etrafındaki sorumluluklar arayüzlere atanır. Böylece birimlerdeki değişiklikler birbirini en az etkiler ve birimler değişikliklerden korunur. Hedef, yapıyı ileride olacak değişikliklere karşı korumaktır.

13. Kalıpların Ardışıl Şemada Gösterilmesi: Kahve Sitesi

> **Hocanın vurgusu:** Etkileşim şemalarında tasarım kalıplarını dikkate alın. Denetçi, uzman ve yaratıcı kalıplarını en iyi gösterebildiğimiz şemalar ardışıl şemalardır. Ardışıl şemaları tasarıma özel hâle getirmek için bu kalıpları içine alabilirsiniz.

Hoca geçen hafta görülen döngü içeren ardışıl şemaya döndü ve kalıpların nerede kullanıldığını gösterdi. Bir öğrencinin sorusu olmadığını söylemesi üzerine yine de şemayı bir kez daha açıkladı.

13.1 Yeni Sipariş

- Dış ortamdan gelen sipariş bilgisini **denetçi** alır.
- Sipariş nesnesi **yaratıcı** kalıbına göre yaratılır.
- Sipariş kalemleri de **yaratıcı** kalıbıyla yaratılır.

13.2 Kahve Ekle (Siparişe Yeni Ürün Ekleme)

- **Denetçi**, dış ortamdan alınan bilgilerin uygunluğunu kontrol eder.

- Seçilen kahvenin bilgisi kahve kataloğundan alınır. Kahve bilgisi katalogda olduğu için burada **bilginin uzmanı** kalıbı devreye girer.
- **Yaratıcı** kalıbıyla bir sipariş kalemi yaratılır.
- Her siparişte hangi kalemlerin olduğu ve kalemlerin listesi toplamda gösterilir.

13.3 Özel Kahve Siparişi

- Bilgi girişini yine **denetçi** yapar.
- Sipariş, **yaratıcı** kalıbının desteğiyle yaratılır.
- Her siparişin içindeki özel kalemler tek tek yaratılır. Bir döngü içinde, kaç tane standart kahve varsa her birinin tanımı alınıp siparişe eklenir.
- Kahve kataloğu ile kahveler arasında ilişki kurulur.

Hocanın verdiği sınav ipucu: Kalıpların görevlerini bilirseniz, örnek olarak verilen bir soruda hangi tasarım kalıbı uygunsa onu yerine yerleştiren bir yapı oluşturabilirsiniz.

14. Etkileşim Şemaları ve Kahve Sitesi Sınıf Şeması

14.1 Ardışıl Şema ile İşbirliği Şeması

İşbirliği şemaları (collaboration diagrams) ile ardışıl şemaların ikisine birlikte etkileşim şemaları (interaction diagrams) denir. Bunlar işlemlerle nesneler arasındaki etkileşimi gösterir.

- **Ardışıl şema:** Nesnelerin olaya hangi sırayla girdiğini belirtir. Bu yüzden “sıralı” ya da “ardışıl” şema denir.
- **İşbirliği şeması:** Her biri bir nesne ya da sınıf olan kutular çizilir. Hocaya göre burada olaya giriş sırası, yani zaman sırası önem taşımaz; şema yalnız sınıflar arasındaki işbirliğini gösterir.

Bu derste işbirliği şeması kullanılmayacak. Tasarım için daha çok sınıf şeması ve ardışıl şema, ayrıca tasarımı tam tanımlamak için etkinlik şemaları ve durum şemaları kullanılacak.

> **Ders sonrası not:** Hoca işbirliği şemasında zaman sırasının önem taşımadığını söyledi; doğrusu sıranın farklı biçimde gösterildiğidir. UML 2’de adı iletişim şeması (communication diagram) olan bu şemada mesajlar numaralanır (1, 1.1, 2 …) ve sıra bu numaralarla verilir. Ardışıl şemadan farkı, sıranın dikey zaman ekseniyle değil numarayla gösterilmesi ve nesneler arasındaki bağlantıların öne çıkmasıdır.

14.2 Kahve Sitesi Sınıf Şeması

Hoca son olarak kahve sitesinin sınıf şemasının genel yapısını açıkladı:

- Kahve sitesi, kahve kataloğundan bilgi alıp gösterir.
- Bilgi girişinin kontrolü denetçi ile yapılır.
- Bir kahve sitesinden çok sayıda sipariş alınabilir, ama en az bir sipariş alınmış olmalıdır.
- Sipariş, fatura ve ödeme ile ilişkilidir.
- Sipariş, sipariş kalemlerinden oluşur. Sipariş ile kahve tanımları arasında kesikli bir ilişki çizgisi vardır.
- Her sipariş kalemi bir kahveyi tanımlar: fiyatı kahveden, miktarı kendisinden alır. Kalem ile kahve arasında en az bir karşılık olmalıdır.
- Standart kalem ve özel kalem, sipariş kaleminden türer.

```
KahveSitesi  1 ---- 1..*  Sipariş
Sipariş      ---- Fatura
Sipariş      ---- Ödeme
Sipariş      1 ---- *  SiparişKalemi
SiparişKalemi  * ---- 1  Kahve          (fiyat Kahve'den, miktar kalemden)
KahveKataloğu  ---- Kahve
StandartKalem  --|>  SiparişKalemi      (genelleştirme)
ÖzelKalem      --|>  SiparişKalemi      (genelleştirme)
```

Hoca standart kalem ve özel kalem ile sipariş kalemi arasındaki ilişkinin türünü sordu; cevap genelleştirme (generalization) oldu.

Örnek Sorular

Soru 1 (derste sözlü, 00:10:53): Gereksinim analizinde pazarlık (negotiation) aşamasında ne yapılır?

Cevap (derste verildi): Müşterinin farklı birimlerinin (örneğin tasarım ve muhasebe) talepleri birbiriyle uyuşmayabilir ya da aynı olayı farklı anlatabilirler. Bazı isteklerin maliyeti çok yüksektir ya da uygulanabilirliği düşüktür. Bu aşamada taraflar iletişim kurar; bazı isteklerden vazgeçilir, bazıları farklı biçimde yapılır. Pazarlık müşteri ile yazılım geliştirici arasında da olur ve yapılamayacak istekler çıkarılır.

Soru 2 (derste sözlü, 00:19:35): Kullanım şemasında include ve extend ilişkileri neyi ifade eder?

Cevap (derste verildi): Include, A işlevini yapabilmek için B işlevinin de her zaman yapılması gerektiğini belirtir; örneğin satış yapmak ödeme almayı içerir, sipariş vermek önceden bilgi girişini içerir. Extend, A işlevi yürütülürken bazı koşullarda B işlevine sapıldığını belirtir; örneğin kartla ödemede provizyon alınır, nakit ödemede gerekmez.

Soru 3 (derste sözlü, 01:11:53): Analiz ile tasarım arasındaki fark nedir?

> **Çözüm (derste verilmedi):** Soruya derste cevap gelmedi ve hoca araya geçti. Dersin kendi anlatımından kurulan cevap: Analiz (çözümleme), problemi anlamak için sistemi parçalara ayırmaktır; uygulama alanının dilini, kavramlarını, çalışanlarını ve işlemlerini öğrenir, yani “ne yapılacak?” sorusunu cevaplar. Çıktısı kullanım şemaları, kullanım senaryoları, kavramsal sınıf şeması ve ER şemalarını içeren SRS belgesidir. Tasarım, bu belgeye dayanarak çözümün “nasıl” kurulacağını belirler: ardışıl, etkinlik ve durum şemaları ile işlemleri de içeren tasarım sınıf şeması çizilir, sorumluluklar tasarım kalıplarına göre atanır.

Soru 4 (derste sözlü, 01:39:09): Çok şekillilik (polimorfizm) nedir?

Cevap (derste kısmen verildi): Öğrencilerden cevap gelmedi. Hoca kalıp olarak şunu söyledi: Davranışın tipe göre değiştiği durumlarda sorumluluk, ilgili tiplere çok şekilli metotlar kullanılarak atanır.

> **Çözüm (derste verilmedi):** Çok şekillilik, aynı mesajın (aynı adlı metot çağrısının) nesnenin gerçek tipine göre farklı davranışlarla karşılanmasıdır. Örneğin derste geçen kredi kartı adaptörlerinde uygulama tek bir “onayla” işlemini çağırır; hangi kuruluşun adaptörü kullanılıyorsa onun gerçeklemesi çalışır.

Soru 5 (derste sözlü, 02:00:04): Kahve sitesi sınıf şemasında standart kalem ve özel kalem ile sipariş kalemi arasında hangi tür ilişki vardır?

Cevap (derste verildi): Genelleştirme (generalization). Standart kalem ve özel kalem, sipariş kaleminden türer.

Dersin Toparlaması

Bu derste nesneye yönelik çözümleme süreci baştan sona tekrar edildi: uygulama alanının anlaşılması, gereksinim analizinin işi anlama, bilgi toplama, modelleme ve pazarlık adımları, kullanım öyküsü belgesi ile kullanım şeması, aday sınıfların bulunması ve kavramsal sınıf şeması. Bu sürecin sonunda SRS belgesi elde edilir ve tasarım bu belgeye dayanır. Tasarım tarafında ardışıl şema kısaca hatırlatıldı; etkinlik şeması (fork, join ve kulvarlar) ile durum şeması uçak ve market kasası örnekleriyle işlendi; tasarım sınıf şemasının kavramsal şemaya işlemleri eklediği gösterildi.

Dersin ikinci yarısında tasarım kalıpları anlatıldı. Dokuz temel kalıbın (yaratıcı, bilginin uzmanı, az bağımlılık, denetçi, yüksek uyum, çok şekillilik, yapay sınıf, dolaylılık, değişimlerden korunma) ortak amacı sorumlulukları doğru atamak, yüksek uyum ve az bağımlılık sağlamak ve yeniden kullanılabilirliği artırmaktır. Kalıplar kahve sitesi ardışıl şemasında gösterildi. Önümüzdeki hafta analiz ve tasarım için, bütün şemaların bir örnek üzerinde oluşturulacağı bir uygulama dersi yapılacak.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.